Journal: known issues

Open findings for the journal, found while writing these documents in 2026-10, and the journal findings fixed earlier. Nothing in the code was changed for this phase; every open row is recorded for the rebuild, and for fixing in the current code if wanted.

Rows marked **tested** were reproduced by running the real code on a scratch database. The others were confirmed by reading the code line by line.

Open

HIGH: the numbers are wrong

#	What	Where
D-J1	**File imports pair fills wrongly.** The first entry fill opens a position at its own price and size; further entries are ignored; the first exit closes the whole position at its own price; later exits are dropped. Any scale-in or scale-out gives the wrong shares, prices, P&L and commission. **Tested**: buy 100 @ 10, buy 100 @ 11, sell 100 @ 12, sell 100 @ 13 is stored as one 100-share trade with net $198, against a real $396	`journal/src/journal/importer.js` `matchTrades`
D-J2	**TradingView fills lose every short.** `sell` is always read as an exit, so a short's opening sale is skipped, and its cover opens a long that never closes and is dropped. The ticker also keeps its exchange prefix (`NASDAQ:AAPL`), so it never matches the same stock from Alpaca. **Tested**	`importer.js` `parseTv` , `isEntry`
D-J3	**Importing a file twice stores every trade twice.** Each trade gets a new random id, so `INSERT OR IGNORE` never ignores anything. **Tested**	`importer.js` `importCsv`

MED

#	What	Where
D-J4	**The TradingView paper log cannot be imported.** Any file with a “Call to place … order” line fails with `Cannot read properties of undefined (reading 'toUpperCase')`: the pattern has three capture groups and the code reads groups 3 and 4. **Tested**	`importer.js` `parseTvLog`
D-J5	**TTP orders history imports nothing.** It is detected by `Date/Time`, `Order ID` and `Event`, then handed to the TTP trades parser, which reads `Date`, `Time` and `Action` columns that are not there. Every row is skipped. **Tested** on a file with the detected headers (no real export was available)	`importer.js` `detectFormat`, `parseTtp`
D-J6	**DXtrade positions closed by hand never close.** Only `Market` fills are entries and only `Stop loss` / `Take profit` fills are exits. A market sell that closes a long is read as a short entry and ignored; limit orders are ignored entirely. The long never closes and is dropped. **Tested**	`importer.js` `parseDxtrade`
D-J7	**A file import drops what it cannot pair, silently.** Positions still open at the end of the file and unmatched exits are thrown away; the answer reports only what was stored. Combined with D-J1 to D-J6, an import can store far less than the file holds and look successful	`importer.js` `matchTrades`, `routes/journal.js` `/import`
D-J8	**The bar window is an hour late from March to November.** The window is written with a fixed −05:00 offset, so in daylight time it starts at 10:30 New York. A trade entered 09:30–10:29 finds no entry bar and the code takes the **last bar of the day** instead, so its snapshot, MAE and MFE describe the close. Same in the Alpaca fallback	`journal/src/journal/technicalFetch.js` `fetchYahooIntraday`, `computeTechnical`; `journal/src/alpaca/client.js` `fetchIntradayBars`
D-J9	**No trade has a stop, so there is no R.** No import or fetch sets one, and the page has no field for it. Every R-based number (expectancy R, SQN, the R distribution), capture %, exit verdict and the whole grading engine are empty	importers, `addons/accounts.js`, `journal.html`, `technicalFetch.js`

#	What	Where
D-J10	**The 📈 chart marks the trade 4 or 5 hours early.** The page builds instants from the New York clock written as UTC and sends them as `entry_ts` / `exit_ts`. qp's print sheet keys bars by real UTC instants, so a 09:45 entry is marked at 05:45 in summer	`journal/public/addons.js` `chartUrl`, `journal.html` `adaptTrade`
D-J11	**An Alpaca position older than the look-back window turns into a phantom trade.** Only fills after now − *days* are read. A long bought before the window and sold inside it arrives as one sell and becomes an open **short**; the real trade, if stored earlier as open, is never closed. Hits every trade held longer than *days* (default 5). **Tested**	`addons/accounts.js` `runOne`, `addons/fillsToTrades.js`
D-J12	**A trade has one date.** `date` is the entry's; exit time is a clock time with no date. For an overnight trade the duration is computed from the two clock times on the entry date (Alpaca and by hand), the technicals fetch only the entry day, and the page orders exits by the entry date. **Tested**: Monday 10:00 → Wednesday 11:30 is stored as 1.5 hours	`journal_trades` schema, `accounts.js` `importTrades`, `routes/journal.js`
D-J13	**The entry snapshot is mostly blank or sees the future.** RVOL is never computed (the volume baseline is never built), nor is ATR; pre-market levels and the market context read `r1_frozen`, which nothing writes in the journal's database; the multi-day levels run on the trade day alone, over **all** its bars, so 2-day/week/month VWAP and highs/lows include bars after the entry, and the 5-day MA and prior day are blank. Measured: of the 77 seeded checks, **55 can never give an answer** for a journal trade	`technicalFetch.js`, `trading/dailyLevels.js`, `trading/volumeBaseline.js`, `trading/historicalCache.js`
D-J14	**The grading half cannot be used.** `trading_setups` has no writer, so a fresh database has no setups and no trade can be tagged. There is no screen for checks, templates or assignments. A card is never updated when a trade's setup changes, and imports are never copied because they arrive untagged. No route or page reads `trade_cards`, and the routes the code points to (`/api/trading/grading/*`) do not exist	`server.js`, `bridgeToGrading.js`, `trading/grading.js`, `trading/checks.js`

LOW

#	What	Where
D-J15	**Fees.** Commission is 0 for Alpaca trades (regulatory fees are not read), DXtrade and the paper log. File imports count only the opening and the first closing fill's commission. Fee profiles are stored and never applied. Commission cannot be edited	`accounts.js`, `importer.js`, `routes/journal.js` `/fees`
D-J16	**Half the server is unused.** 14 of 29 routes have no caller (one trade, hand entry, technicals, context, templates, notes, fees, the server's metrics and calendar, the backfill). The database also carries the old app's screener tables and the removed live engine's tables and 46 settings, of which the journal reads two (the Alpaca key and secret, as a fallback for bars). Two condition systems exist side by side (`alignment.js` with templates, `checks.js` with the library); nothing reads the first	`journal/src/db/index.js`, `trading/db.js`, `routes/journal.js`
D-J17	**The server's numbers differ from the page's.** `/analysis/metrics` computes expectancy as win rate × average win − (1 − win rate) × average loss, which puts 0 trades on the loss side; the page uses net ÷ closed. `/analysis/calendar` ignores the account	`journal/src/journal/analysis.js`
D-J18	**Out-of-date or wrong text.** The code repo's `docs/USER-GUIDE.md` journal table puts the R distribution on the Time tab (it is on Risk), the Alpaca accounts on the Import tab (they are on Trades), and "notes, tags, fees" on Trades (only the setup can be changed). The registry's description promises R-multiple, MAE/MFE and notes, which the page never shows. The Import hint calls the semicolon format "Trade the Pool" (it is DXtrade). The SQN legend says √N, but the code caps N at 100. The import message looks for `skipped` and `format`, which the server never sends	`docs/USER-GUIDE.md`, `tools.config.json`, `journal.html`
D-J19	**Missing data reads as "not aligned"** in five template rules (`price_vs_vwap`, `price_vs_ema9/13/20/50`): they compare a missing value with `above` / `below` and get false, where the other rules give blank	`journal/src/journal/alignment.js`
D-J20	**Small display faults.** The P&L histogram drops a trade whose loss equals the largest absolute P&L; a 0 trade does not break a streak; trades dated on a weekend are left out of the calendar and its month	`journal.html`, `importer.js`

#	What	Where
	total; a file-imported short’s % move is not signed by direction	

Fixed before this review

Six of the eight fixes landed with a test in the code repo; the other two are recorded in the code beside the fix.

#	What was wrong	Test
J-1	The metrics, calendar and compute-all queries wrote `"closed"` in double quotes, which SQLite read as a column name: all three answered 500 on every call	`journal.closedQueries.test.js`
J-2	The grading backfill threw on the first trade with no setup (the card’s setup is required) and ended the whole run	`journal.robust.test.js`
J-3	Bars could not be fetched from a database with no `trading_brokers` table; every compute answered 500	`journal.robust.test.js`
J-4	Tagging a setup asked for an `id` column the snapshot table did not have, so the condition step after it failed. The route now asks for `trade_id`, and the column is also added at start-up, after the table exists	none: the repair and its reason are recorded in `journal/server.js`
J-5	The calendar put days under the wrong weekday: the week total was written twice, shifting the month a column per week	`journal.calendar.test.js`
J-6	Deleting a trade could fail silently: the browser dialog is blocked on some phones, and the reply was never checked. Replaced by a two-tap confirm that reports failure	none: the fix and its reason are recorded in `addons.js`
J-7	The journal had no way out: its “← Dashboard” link pointed to itself. It now carries the desk’s app bar	`journal.appbar.test.js`
J-8	There was no setup filter, and the account filter missed Alpaca accounts with no trades yet and lost them again on every Refresh	`journal.accounts.test.js`